

Carnegie
Mellon
University

Recitation

Debugging Deep Learning Models

Introduction to Deep Learning | 11-485 / 685 / 785

Debugging by elimination, not by guessing

Felix Hirwa Nshuti

Fall 2026

The problem is silent failure

Deep learning bugs do not crash. That is what makes them expensive.

In ordinary software, a bug usually crashes and the stack trace names the line. The bugs that cost you a week in this course produce a running program, plausible tensors, and a loss curve that goes down.

Three real examples, all of which train without complaint:

1 Labels off by one

Forgot `y[1:-1]` on the HW1P2 transcript. Trains normally. Caps around 40% accuracy and never tells you why.

2 Softmax applied twice

Softmax before `CrossEntropyLoss`. Initial loss is still correct. Gradients are compressed, so it learns, just far too slowly.

3 Stats from the wrong split

Normalization fitted on validation. Validation accuracy looks great. Test accuracy collapses and you blame the architecture.

Two kinds of failure, and only one is hard

Fix every bug of the first kind before forming any hypothesis about the second.

Class A: it crashes

- Shape mismatch, device mismatch, dtype error, CUDA out of memory.
- The stack trace localizes it. Read it bottom up, print shapes, fix, move on.
- Annoying. Not hard. Never the reason a model fails to learn.

Class B: it runs and does not learn

- No trace, no warning, no line number. The program is behaving as written.
- Loss is flat, or falls to a plateau far above what the task allows.
- This is the entire subject of the next eighteen minutes.

The ordering rule

Coding errors first, training errors second. If anything still raises an exception, you are not allowed to have an opinion about your learning rate yet. Every hypothesis you form about optimization while a Class A bug is live is a hypothesis about a program you are not actually running.

Debugging is elimination

Four steps. The discipline is in step three.

- 1 State a falsifiable claim**

Not "something is wrong with my data". Instead: "the Dataset returns labels misaligned by one frame". A claim you cannot test is not a hypothesis, it is a mood.
- 2 Design the cheapest falsifying test**

Cheapest is doing real work here. The cheapest test is usually not the first one you thought of. Printing two labels beats retraining for an epoch.
- 3 Change exactly one thing**

One variable per run. If you must change two, you have two tests. This is the step everyone skips and it is the one that makes the others worthless.
- 4 Write down the result**

Hypothesis, test, outcome, kept or reverted. Four columns. Without it you re-run experiments you already ran.

The anti-pattern: changing learning rate, batch size and architecture in one commit, then not being able to say which one mattered.

The four buckets

Assume the bug lives in exactly one of these. Never debug two at once.

Bucket	Common failure modes
Data	Label and input mismatch, normalization differing between train and test, leakage across splits, shuffling applied to the wrong split.
Model	Architecture and parameter mismatch, bad initialization, accidentally frozen parameters after loading a checkpoint, layers built outside <code>__init__</code>
Loss	Objective does not match the task, activation applied twice, wrong reduction, wrong target format or dtype.
Training loop	Missing <code>optimizer.zero_grad()</code> , missing <code>optimizer.step()</code> , <code>backward()</code> on the wrong quantity, scheduler stepped per batch instead of per epoch.

One bucket at a time, but how?

"Do not debug two at once" is only actionable if you can isolate one bucket from the other three. The mechanism is known-good stubs, and it is on slide twelve. Everything between here and there is about deciding which bucket to open first.

Symptom in, bucket out

The buckets tell you where bugs live. This tells you which one to open.

What you observe	Open this bucket first
Loss is exactly constant and never moves	Training loop. Missing zero_grad or step, lr = 0, or every parameter frozen.
Loss at step 0 is far from $\ln(C)$	Loss. Wrong target format, wrong reduction, or an activation before the criterion.
Loss falls but accuracy stays flat	Model or metric. argmax on the wrong dim, or labels misaligned with features.
Loss becomes NaN or inf after N steps	Loss, then Model. Learning rate too high, $\log(0)$, or missing normalization.
Train loss falls, val flat or rising from step 1	Data. Leakage, normalization mismatch, or a broken split.
Validation is better than training	Data or Model. Dropout still active in eval, or stats fitted on val.
Cannot overfit eight samples	Everything upstream of optimization. Go to slide 8 and do not touch a hyperparameter.

None of these are certainties. They are the highest-probability first move, which is all a search procedure has to give you. Being wrong cheaply and quickly beats being thorough slowly.

CHECK 1

Expected initial loss

One number, checked before you train at all.

```
import math, torch.nn.functional as F

with torch.no_grad():
    logits = model(x[:256])
    print(F.cross_entropy(logits, y[:256]).item(), math.log(NUM_CLASSES))
```

C = 2

0.693

BCE

C = 10

2.303

CE

C = 40

3.689

HW1P2

C = 1000

6.908

CE

If it matches $\ln(C)$

- Output layer width, target format, dtype and reduction are all probably right.
- You have ruled out a large part of the Loss bucket in about four seconds.

If it does not

- Much lower: leakage, or a checkpoint you forgot you loaded.
- Much higher: logits are enormous. Bad init, or missing input normalization.
- Exactly $\ln(C)$ and staying there: route to the Training loop bucket.

Overfit a tiny batch

If the pipeline cannot memorize eight samples, nothing downstream matters.

```
xs, ys = x[:8], y[:8]                # eight samples, fixed, never shuffled
model.train(); disable_dropout(model) # dropout off, augmentation off, wd = 0

for step in range(300):
    opt.zero_grad()
    loss = criterion(model(xs), ys)
    loss.backward(); opt.step()
    if step % 50 == 0: print(step, loss.item())
```

If loss reaches roughly zero

- Forward, backward, loss and optimizer are wired correctly.
- Your problem is generalization, data scale or capacity. Not plumbing.
- It should also get there fast. If memorizing eight samples takes 5000 steps, your gradients are compressed: suspect a doubled activation.

If it plateaus or stays flat

- The bug is upstream of optimization. No learning rate schedule will rescue this.
- Nothing about your architecture is worth changing yet.
- Go straight to stubs on slide 12 and bisect.

Assert your shapes at boundaries

Broadcasting is the mechanism by which wrong code keeps running.

```
def forward(self, x):  
    B = x.shape[0]  
    assert x.shape == (B, self.in_dim), f'input {tuple(x.shape)}'  
    out = self.net(x)  
    assert out.shape == (B, self.num_classes), f'output {tuple(out.shape)}'  
    return out                                     # raw logits, no activation
```

A tensor of shape (B, 1) and a tensor of shape (B,) combine into (B, B) without a word of complaint. The loss computes, gradients flow, every number is wrong.

What you get

- Failures move from silent to loud, and land at the boundary you crossed.
- The assertion message carries the actual shape, so the fix is usually immediate.

What it costs

- Asserts run in the hot path. Negligible next to a matmul, but gate them if you are measuring throughput.
- `python -O` strips asserts entirely. Do not rely on them for correctness.

CHECK 4

Audit your gradients

Three different states that all look like "no learning".

```
loss.backward()
total = 0.0
for name, p in model.named_parameters():
    if p.grad is None:
        print(f'{name}: NO GRAD <- not in the graph at all'); continue
    g = p.grad.norm().item(); total += g * g
    if g == 0: print(f'{name}: zero gradient')
print(f'total grad norm: {total ** 0.5:.4e}')
```

What you see	What it means
p.grad is None	The parameter is not in the graph. requires_grad is False, it was detached, or the layer was built after the forward pass ran.
p.grad.norm() == 0	In the graph, receiving no signal. Dead ReLU, a mask that zeroes everything, or a multiply by zero upstream.
norm is inf or NaN	Exploding gradients, or log(0) and division by zero inside the loss. Go to slide 17.
norm is tiny but nonzero	Vanishing signal, or an activation applied before the criterion compressing the gradient.

Make the bug reproducible

A bug you cannot reproduce is a bug you cannot bisect.

```
random.seed(0); np.random.seed(0); torch.manual_seed(0)
torch.cuda.manual_seed_all(0)
torch.use_deterministic_algorithms(True)    # some ops now RAISE
torch.backends.cudnn.benchmark = False      # slower, but stable

g = torch.Generator(); g.manual_seed(0)     # seeding Python is NOT enough:
DataLoader(ds, shuffle=True, generator=g,    # the loader has its own RNG,
           num_workers=0)                   # and workers have theirs
```

What you get

- The same failure, every run, so a fix is distinguishable from luck.
- Two runs that differ now mean you changed something, not that the seed moved.
- Combine with dropout off and augmentation off to remove the rest of the noise.

What it costs

- `use_deterministic_algorithms` makes some ops raise instead of run. You may have to work around one.
- Determinism and `cudnn.benchmark = False` are both slower. Turn them off for real training runs.
- Seeds do not propagate to workers without `worker_init_fn`. Use `num_workers = 0` while debugging.

Isolation by stubs

This is the mechanism behind "one bucket at a time".

```
# Replace one stage with something known to be correct, then re-run.

x, y = torch.randn(64, D), torch.randint(0, C, (64,)) # DATA stub
model = nn.Linear(D, C).to(device) # MODEL stub
criterion = nn.CrossEntropyLoss() # LOSS stub
# LOOP stub: one batch,
# one step, print loss
# before and after
```

Failure disappears with the stub in

- The bug is in the stage you just replaced. You have localized it to one bucket.
- Example: a plain `nn.Linear` trains on your real data but your network does not. The bug is in your architecture, not your pipeline.

Failure survives the stub

- The bug is downstream of the stage you replaced. Move the stub one stage later and repeat.
- Example: random data with random labels still will not overfit. Data is exonerated. Look at loop and loss.

Keep a bug ledger

Four columns in a text file. It is not bureaucracy, it is memory.

Hypothesis	Test	Result	Kept?
Labels misaligned by one frame	print x[64], y[64], y[65] for one utterance	y[65] matched. Off by one confirmed.	fixed
Lr too high, causing the NaN	rerun 200 steps at Lr = 1e-4	still NaN at step 143	reverted
NaN comes from log(0) in the loss	set_detect_anomaly, read the trace	traced to a masked log. Confirmed.	fixed
Model too small for the task	widen 1024 to 2048, 3 epochs	val accuracy 61.2 to 61.4. Noise.	reverted

Why it earns its keep

Three hours in, you will not remember whether you already tried the thing you are about to try. The ledger is also what turns an office hours visit from twenty minutes of reconstruction into two minutes of reading.

One change per row. If a row describes two changes, it is two rows and you have not learned anything from either.

Log five scalars, every step

Cheap, and between them they localize most failures.

loss

The obvious one. Log per step, not per epoch: an epoch-level curve hides a divergence that happened in the first fifty steps.

total gradient norm

The first derivative of "is anything happening". Flat loss plus zero grad norm is a very different bug from flat loss plus healthy grad norm.

current learning rate

Read it from the optimizer, not from your config. A scheduler stepped per batch instead of per epoch drives lr to zero within one epoch and your config still says $1e-3$.

logit standard deviation

Collapsing toward zero means a dead branch in the graph. Growing without bound precedes a NaN by a few hundred steps.

If you only log one extra thing beyond loss, log the gradient norm.

Interactive inspection with breakpoint()

An entry point, not a full tour. The class labs go deeper on this.

```
def __getitem__(self, i):
    x = self.padded[i : i + self.width]
    breakpoint()          # modern form of pdb.set_trace(), Python 3.7+
    return torch.from_numpy(x.reshape(-1)), self.labels[i]

# (Pdb) p x.shape, x.dtype, x.device      <- the workhorse command
# PYTHONBREAKPOINT=0 python train.py     <- disable every breakpoint, no edits
```

The eight commands that cover almost everything:

n next line

s step into call

c continue

l list source here

w where am I (stack)

p / pp print / pretty-print

u / d up / down a frame

interact full REPL at this frame

Optional to explore now. If you want to go further, the class labs cover interactive debugging properly.

Where breakpoint() will not help you

Three traps, one of which wastes an entire afternoon.

1 DataLoader workers

With `num_workers > 0`, a breakpoint inside `__getitem__` lands in a subprocess with no stdin. It hangs, or dies on EOF.

Fix: `num_workers = 0` while you are debugging the Dataset.

2 Inside `no_grad()`

Your tensors have no `grad_fn` and `requires_grad` is False.

That is correct behaviour, not a bug. People spend twenty minutes "fixing" it. Check where you are first with `w`.

3 Code you submit

A breakpoint left in a file sent to Gradescope hangs the grader until it times out.

Use `PYTHONBREAKPOINT=0` to disable globally, and `grep` for breakpoint before you submit.

The general point

Every one of these is the debugger telling you the truth about a context you did not realize you were in. Before concluding the tool is broken, run `w` and confirm which process and which frame you are actually stopped in.

Localizing a NaN

The one case where the default trace points nowhere useful.

```
torch.autograd.set_detect_anomaly(True)    # localize only, then turn OFF

# backward now raises AT the op that produced the NaN, and the traceback
# points back to the FORWARD line that created that op.
```

What it buys you

- Without it, backward reports a NaN somewhere with no forward context.
- With it, you get the exact forward line. This usually ends the investigation immediately.
- Pair it with the gradient audit on slide 10 to see how far the NaN has spread.

What it costs, and where NaNs come from

- Substantially slower. Localize, fix, then turn it off. Never leave it on for a real run.
- Usual sources: $\log(0)$, division by zero, sqrt of a negative, learning rate too high, fp16 overflow.
- If the NaN appears at a consistent step count, suspect the data at that index, not the math.

Hooks, when printing is not enough

Instrument a layer without editing its forward().

```
def stat(name):  
    def hook(module, inp, out):  
        print(f'{name}: mean {out.mean():.4f} std {out.std():.4f} '  
              f'dead {(out == 0).float().mean():.1%}')  
    return hook  
  
for name, m in model.named_modules():  
    if isinstance(m, nn.ReLU):  
        m.register_forward_hook(stat(name))    # remove() when you are done
```

What to look for

- The layer where activation std collapses toward zero. Everything after it is receiving noise.
- A dead fraction near 100% on a ReLU means that block contributes nothing at all.
- Works on any nn.Module without touching the model code, which matters for code you did not write.

What it costs

- Hooks fire on every forward. Register, run a few batches, then remove them.
- A hook that holds a reference to out will keep that activation alive and leak memory across steps.
- Print volume gets out of hand fast. Gate on a step counter.

Bug gallery: it runs and does not learn

The ones we see every semester, keyed by symptom.

Symptom	Cause	Bucket
Loss pinned at exactly 3.689 forever	Missing optimizer.step(), or lr = 0, or every parameter frozen after loading a checkpoint	Loop
Loss decreases, accuracy caps near 40%	Forgot y[1:-1]. Every label is offset by one frame	Data
Loss decreases absurdly slowly	Softmax applied before CrossEntropyLoss. Initial loss looks correct, gradients are compressed	Loss
Loss decreases for 2 epochs then NaN	Learning rate too high, or log(0) on a masked value. Use anomaly mode	Loss
Model predicts a single class for everything	Collapse. lr too high early, or severe class imbalance with no weighting	Loss / Data
Gradient norm is exactly 0 at every layer	A multiply by zero or a mask that zeroes the whole batch upstream	Model
p.grad is None for half the network	Those layers were built outside __init__, or requires_grad was left False	Model

Read the first column, not the second

This table is deliberately organized the way you experience the bug, not the way you would file it afterwards. When you hit something not on this list, the routing table on slide six is the general version.

Bug gallery: it learns, and the numbers lie

Worse than not learning, because you might submit it.

Symptom	Cause	Bucket
Validation accuracy above training accuracy	<code>model.eval()</code> never called, so dropout is still active during training measurement	Loop
Great validation, collapsed test	Normalization statistics fitted on validation, or leakage across the split	Data
Loss falls smoothly, accuracy random	<code>argmax(dim=0)</code> instead of <code>argmax(dim=1)</code> in the metric, not in the model	Metric
CUDA OOM during validation only	Missing <code>torch.no_grad()</code> , so the graph is retained for every val batch	Loop
Results change every run by several points	Nothing seeded. You are measuring variance and calling it improvement	Loop
Kaggle score far below local validation	Test preprocessing differs from train, or the submission rows are misordered by a shuffled test loader	Data

The last row is worth a specific warning: never pass `shuffle=True` to a test `DataLoader`. Your predictions will be internally consistent and in the wrong order, and the score will look like a modelling failure.

Every entry here produces plausible numbers. That is exactly what makes them dangerous.

Before you post on Piazza

Six lines. They resolve most questions before you ask them.

- Does the model overfit eight samples to near-zero loss?
- Is the initial loss within a few percent of $\ln(C)$?
- Is the total gradient norm nonzero, finite, and changing?
- Are seeds fixed, dropout off, augmentation off, `num_workers = 0`?
- Have you asserted the shape of every tensor entering and leaving the model?
- Can you state, in one sentence, which of the four buckets you have ruled out and how?

When you do post, include the answers. It turns a thread into one reply.

When confused, simplify

The single most reliable move in this entire talk.

Every technique here is a way of making the failing case smaller, more deterministic, and faster to run. Get to a failure you can reproduce in ten seconds, then debug that. Debugging a 40-minute epoch is not debugging, it is waiting.

1 Smaller

Eight samples. One batch.
One utterance.
The smallest input that still reproduces the failure.

2 Simpler

One linear layer. No dropout,
no augmentation, no scheduler,
no mixed precision.
Add complexity back one piece
at a time.

3 Deterministic

Seeds fixed. `num_workers = 0`.
Shuffling off.
So that the second run tells you
something about your fix.

4 Faster

Ten-second iterations, not
forty-minute epochs.
The cost of one hypothesis is
what limits how many you can
test.

Where to go next

The first two are the canonical references. Read both.

- **Karpathy, A Recipe for Training Neural Networks** karpathy.github.io/2019/04/25/recipe/ - the source of the overfit-a-tiny-batch discipline
- **PyTorch debugging documentation** [pytorch.org/docs/stable/notes/...](https://pytorch.org/docs/stable/notes/) and the autograd anomaly detection reference
- **`torch.autograd.set_detect_anomaly`** pytorch.org/docs/stable/autograd.html#anomaly-detection
- **Recitation 0, Part 3** The data side of this talk: measuring the loader, and why `num_workers = 0` while debugging
- **pdb documentation** docs.python.org/3/library/pdb.html - optional now, covered properly in the class labs
- **Piazza** Post the six checklist answers from slide 21 with your question

Karpathy's recipe is generic and excellent. Slides 19 and 20 are the part that is specific to this course.

Thank you

Your model is not a black box. It is a program with a bug in one of four places.

Reproduce it. Shrink it. Bisect it. Write down what you learned.

Questions on Piazza | Recitation materials on the course site

Carnegie Mellon University